

Министерство образования Республики Беларусь

Учреждение образования
БЕЛОРУССКИЙ ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ
ИНФОРМАТИКИ И РАДИОЭЛЕКТРОНИКИ

Факультет компьютерных систем и сетей

Кафедра информатики

Дисциплина: Операционные среды и системное программирование

ОТЧЕТ
к лабораторной работе №4
на тему

**УПРАВЛЕНИЕ ПРОЦЕССАМИ И
ВЗАИМОДЕЙСТВИЕ ПРОЦЕССОВ**

Студент
Преподаватель

Е. С. Кахновский
Н. Ю. Гриценко

Минск 2024

СОДЕРЖАНИЕ

1 Цель работы	3
2 Теоретические сведения	4
3 Результат выполнения	5
Заключение	5
Список использованных источников	7
Приложение А (обязательное) Листинг кода	8

1 ЦЕЛЬ РАБОТЫ

Изучение основных особенностей подсистемы управления процессами и средств взаимодействия процессов в *Unix*. Практическое проектирование, реализация и отладка программных комплексов из нескольких взаимодействующих процессов.

2 ТЕОРЕТИЧЕСКИЕ СВЕДЕНИЯ

Процесс в *Linux* (как и в *UNIX*) – это программа, которая выполняется в отдельном виртуальном адресном пространстве. Когда пользователь регистрируется в системе, автоматически создается процесс, в котором выполняется оболочка (*shell*), например, */bin/bash*.

В *Linux* поддерживается классическая схема мультипрограммирования. *Linux* поддерживает параллельное (или квазипараллельное при наличии только одного процессора) выполнение процессов пользователя. Каждый процесс выполняется в собственном виртуальном адресном пространстве, т.е. процессы защищены друг от друга и крах одного процесса никак не повлияет на другие выполняющиеся процессы и на всю систему в целом. Один процесс не может прочесть что-либо из памяти (или записать в нее) другого процесса без "разрешения" на то другого процесса. Санкционированные взаимодействия между процессами допускаются системой.

Ядро предоставляет системные вызовы для создания новых процессов и для управления порожденными процессами. Любая программа может начать выполняться только если другой процесс ее запустит.

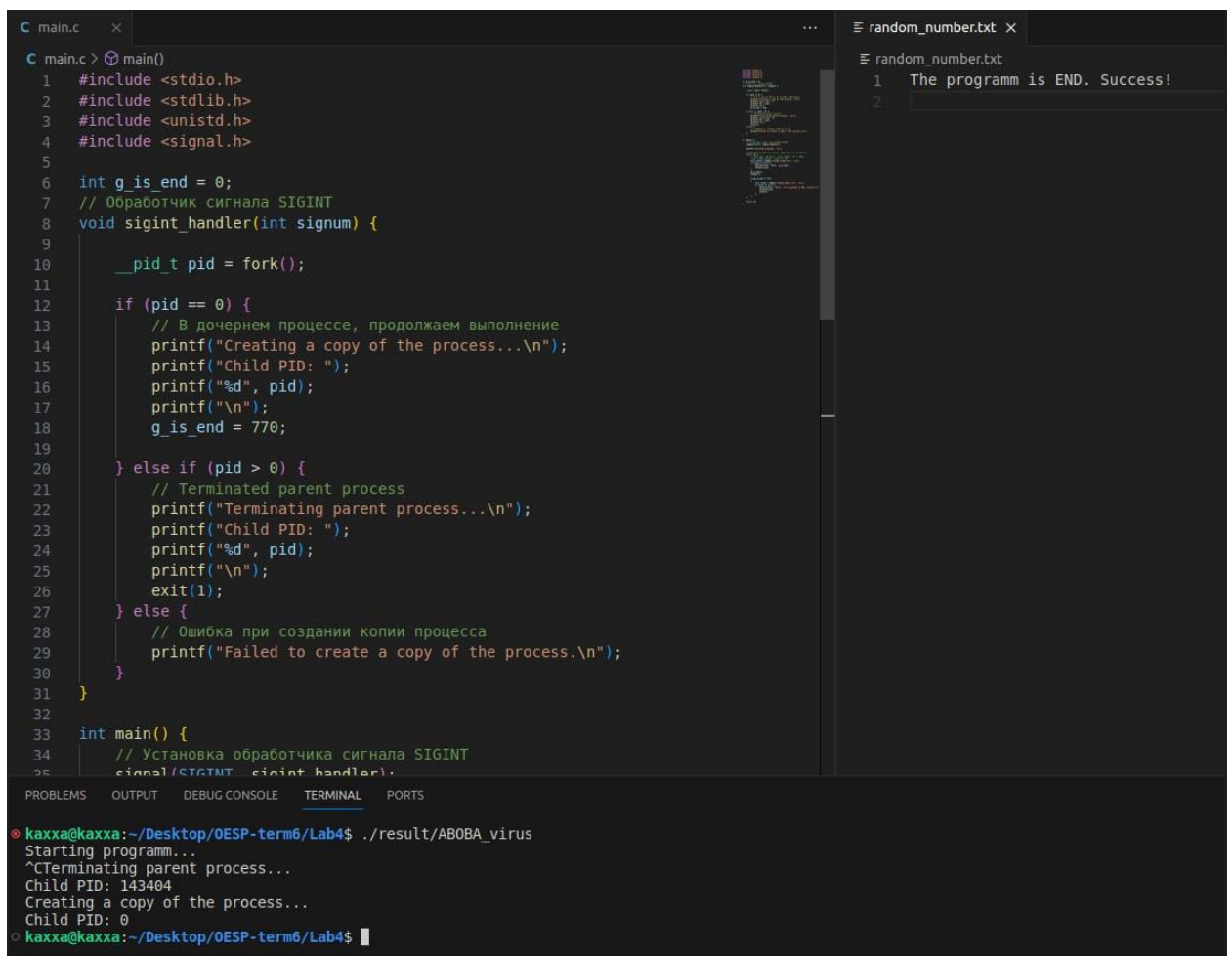
Для создания процессов используются два системных вызова: *fork* и *exec*. *Fork* создает новое адресное пространство, которое полностью идентично адресному пространству основного процесса. Функция *fork* возвращает 0 в порожденном процессе и *PID* (*Process ID* – идентификатор порожденного процесса) – в основном. *PID* – это целое число. Теперь, когда процесс уже создан, можно запустить программу с помощью вызова *exec*. В адресное пространство порожденного с помощью *fork* процесса будет загружена новая программа и ее выполнение начнется с точки входа (адрес функции *main*) [1].

Каналы – неименованные (*pipe*) и именованные (*fifo*) – это средство передачи данных между процессами. Можно представить себе канал как небольшой кольцевой буфер в ядре операционной системы. С точки зрения процессов, канал выглядит как пара открытых файловых дескрипторов – один на чтение и один на запись (можно больше, но неудобно). Мы можем писать в канал до тех пор, пока есть место в буфере, если место в буфере кончится – процесс будет заблокирован на записи. Можем читать из канала пока есть данные в буфере, если данных нет – процесс будет заблокирован на чтении. Если закрыть дескриптор отвечающий за запись, то попытка чтения покажет конец файла. Если закрыть дескриптор отвечающий за чтение, то попытка записи приведет к доставке сигнала *SIGPIPE* и ошибке *EPIPE* [2].

3 РЕЗУЛЬТАТ ВЫПОЛНЕНИЯ

В результате лабораторной работы была создана программа, соответствующая схеме «самовосстанавливающийся процесс». Задача – создать процесс, который при получении сигнала, стандартно вызывающего завершение, создает свою копию, которая продолжает выполняться с прерванного места, и лишь после этого завершается, избегая таким образом безусловного «уничтожения» не перехватываемым сигналом.

После запуска программы, можно наблюдать, процесс записи в файл значения счетчика. Когда основной процесс будет прерван, создается дочерний процесс, который будет инкрементировать счетчик еще некоторый промежуток времени, после чего, программа завершится, выводя в файл результат об успешном завершении программы (рисунок 1).



```
C main.c x
C main.c > main()
1 #include <stdio.h>
2 #include <stdlib.h>
3 #include <unistd.h>
4 #include <signal.h>
5
6 int g_is_end = 0;
7 // Обработчик сигнала SIGINT
8 void sigint_handler(int signum) {
9
10     _pid_t pid = fork();
11
12     if (pid == 0) {
13         // В дочернем процессе, продолжаем выполнение
14         printf("Creating a copy of the process...\n");
15         printf("Child PID: ");
16         printf("%d", pid);
17         printf("\n");
18         g_is_end = 770;
19
20     } else if (pid > 0) {
21         // Terminated parent process
22         printf("Terminating parent process...\n");
23         printf("Child PID: ");
24         printf("%d", pid);
25         printf("\n");
26         exit(1);
27     } else {
28         // Ошибка при создании копии процесса
29         printf("Failed to create a copy of the process.\n");
30     }
31 }
32
33 int main() {
34     // Установка обработчика сигнала SIGINT
35     signal(SIGINT, sigint_handler);
36
37     while (1) {
38         printf("Counting...\n");
39         sleep(1);
40     }
41 }
42
43 random_number.txt x
random_number.txt
1 The programm is END. Success!
2

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
kaxxa@kaxxa:~/Desktop/OESP-term6/Lab4$ ./result/ABOBA_virus
Starting programm...
^CTerminating parent process...
Child PID: 143404
Creating a copy of the process...
Child PID: 0
kaxxa@kaxxa:~/Desktop/OESP-term6/Lab4$
```

Рисунок 1 – Результат работы программы

ЗАКЛЮЧЕНИЕ

В ходе выполнения лабораторной работы были изучены основные особенности подсистемы управления процессами и средств взаимодействия процессов в *Unix*. Практическое проектирование, реализация и отладка программных комплексов, состоящих из нескольких взаимодействующих процессов, были рассмотрены на примере программы, реализующей схему «самовосстанавливающийся процесс».

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

[1] Управление процессами в Linux [Электронный ресурс]. – Режим доступа: https://www.opennet.ru/docs/RUS/lnx_process/process2.html.

[2] Каналы (pipe, fifo) [Электронный ресурс]. – Режим доступа: <https://parallel.uran.ru/node/464>.

ПРИЛОЖЕНИЕ А

(обязательное)

Листинг кода

Листинг 1 – Файл *main.c*

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <signal.h>
int g_is_end = 0;
// Обработчик сигнала SIGINT
void sigint_handler(int signum) {
    __pid_t pid = fork();
    if (pid == 0) {
        // В дочернем процессе, продолжаем выполнение
        printf("Creating a copy of the process...\n");
        g_is_end = 770;
    } else if (pid > 0) {
        exit(1);
    } else {
        // Ошибка при создании копии процесса
        printf("Failed to create a copy of the process.\n");
    }
}

int main() {
    signal(SIGINT, sigint_handler);
    printf("Starting programm...\n");
    while (1) {
        // Генерация случайного числа и запись его в файл
        //int random_number = rand() % 100;
        FILE *file = fopen("random_number.txt", "w");
        if (file != NULL) {
            fprintf(file, "%d\n", g_is_end);
            fclose(file);
        }
        g_is_end++;
        sleep(1);
        if(g_is_end == 777)
        {
            FILE *file = fopen("random_number.txt", "w");
            if (file != NULL) {
                fprintf(file, "%s\n", "The programm is END. Success!");
                fclose(file);
                exit(1);} } }return 0;}
```

Листинг 2 – Файл *makefile*

```
CC = gcc
TARGET = ABOBA_virus
BUILD_DIR = result
$(TARGET): main.o
    $(CC) $(BUILD_DIR)/main.o -o $(BUILD_DIR)/$@
main.o: main.c
    $(CC) -c $< -o $(BUILD_DIR)/$@

clean:
    rm -f $(BUILD_DIR)/*
```